

8.f. Multi – level Linked List – Destroy List(Main List and Child)

–Growth Rate of Output

– Discussion

Program:

```
void destroyNodeRecursively(Node *node)
{
    if (node == nullptr)
        return;

    // 1. Destroy Child List first
    destroyNodeRecursively(node->child);

    // 2. Destroy Next List
    destroyNodeRecursively(node->next);

    // 3. Free current
    cout << "Freeing node: " << node->data << endl;
    free(node);
}

void destroyList()
{
    destroyNodeRecursively(head);
    head = nullptr;
    cout << "Entire list destroyed." << endl;
}

int main()
{
    int choice;
    while (true)
    {
        cout << "\n--- Multilevel List Menu ---" << endl;
        cout << "8. Destroy List" << endl;
        cin >> choice;
```

```

        switch (choice)
        {
        case 8:
            destroyList();
            break;
        default:
            cout << "Invalid choice" << endl;
        }
    }
    return 0;
}

```

Growth Rate of Output

Number of nodes freed (Output Growth)

Let $F(n)$ = number of nodes freed.

Each call eventually frees:

- ***all nodes in the child subtree,***
- ***all nodes in the next subtree,***
- ***the current node itself.***

Therefore

$$F(n) = F(c) + F(r) + 1$$

where,

- ***c = number of nodes in the child substructure,***
- ***r = number of nodes in the next substructure.***
- ***1 = current node itself.***

1. $F(Node \rightarrow child)$ [The Left /Down Branch]:

This term represents the computer

diving into the first recursive call ($destroyNodeRecursively(node \rightarrow child)$).

It mathematically counts every single node that gets successfully freed in the entire child sub – tree before the computer returns to the current node.

2. $F(Node \rightarrow next)$ [The Right /Across Branch]:

This term represents the second recursive call

($destroyNodeRecursively(node \rightarrow next)$).

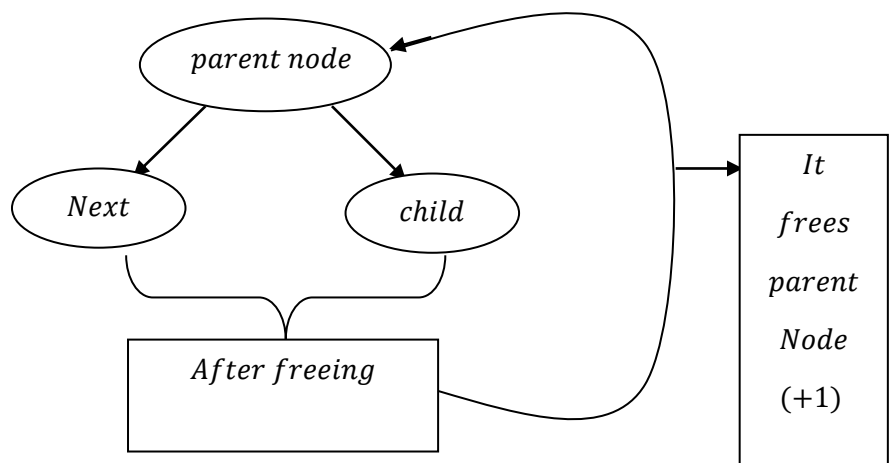
It counts every single node freed in the neighbor/ nextsub – tree.

The computer will not proceed until this entire branch is cleared.

3. $+ 1$ [The Current Node]:

This represents the base operation of the algorithm: $free(node)$.

Once the computer has successfully destroyed both the child branch and the next branch, it turns its attention to the specific node it is currently sitting on. The $+ 1$ is the mathematical tally for destroying the parent node itself.



Therefore , we can also write the equation as:

$$F(Node) = F(Node \rightarrow child) + F(Node \rightarrow next) + 1$$

If we assume the recursive calls correctly count the nodes in their respective sub – trees, we can substitute them with pure variables:

- $F(Node \rightarrow child) = N_{child}$
- $F(Node \rightarrow next) = N_{next}$

Substitute these back into the original equation:

$$F(Node) = N_{child} + N_{next} + 1$$

Because $N_{child} + N_{next} + 1$ is the exact structural definition of N , the equation resolves to:

$$F(Node) = N$$

Also,

$$F(n) = F(c) + F(r) + 1$$

$$\Rightarrow n = c + r + 1$$

$$\therefore F(n) = n$$

The output equation resolves exactly to N . This mathematically proves that if a data structure contains N nodes, the algorithm will execute the `free()` command exactly N times. Because the time spent (operations) maps 1: 1 with the output (nodes freed), the overall Time Complexity of the destruction algorithm is strictly $O(N)$.
